

BRAC UNIVERSITY
Department of Computer Science and Engineering

Set: B

Examination: Final Examination
Duration: 2 Hours

Semester: FALL 2025
Full Marks: 30

CSE 420: Compiler Design

Answer the following questions. Theoretical questions should be answered in a precise, concise, and to-the-point manner. Figures in the right margin indicate marks.

Name:	ID:	Section:
-------	-----	----------

- 1.** Explain, with an example, how a compiler resolves class functions in two classes when they have the same parameters and one class inherits from the other. Furthermore, explain how these functions are correctly called/invoked in the executable code and how function access modifiers impact function calls if they have some impact. 3
- CO5**
- 2.** Explain what the register stack pointer(top-sp) represents in a run-time stack. Why is it placed at the end of the fixed-length fields in an activation record? 2
- CO5**
- 3.** Consider the following SDT. Draw the Annotated Parse Tree, Evaluate the Tree and Write the three address code for the string: $a[x][y] = y + c$; (Annotate the attributes “addr”, “type”, “array” in the parse tree; Finally show the generated three address code) 5
- CO4**

Note : Here “a” is a 2 dimensional integer array. The type of a is array(3, array(2, integer)). Its width w is 24, assuming that the width of an integer is 4. The type of a[x] is array(2, integer), of width w = 8. The type of a[x][y] is integer. The gen() function generates strings and the Temp() class creates a temporary variable which is ‘t.’ with a subscript 1, 2, 3 and so on.

SDT:

```
S → id = E ; { gen(top.get(id.lexeme) '=' E.addr); }

| L = E ; { gen(L.addr.base '[' L.addr ']' '=' E.addr); }

E → E1 + E2 { E.addr = new Temp(); gen(E.addr '=' E1.addr '+' E2.addr); }

| id      { E.addr = top.get(id.lexeme); }

| L      { E.addr = new Temp(); gen(E.addr '=' L.array.base '[' L.addr ']'); }

L → id [ E ] { L.array = top.get(id.lexeme); L.type = L.array.type.elem;

               L.addr = new Temp(); gen(L.addr '=' E.addr '*' L.type.width); }

| L1 [ E ] { L.array = L1.array; L.type = L1.type.elem;

               t = new temp(); L.addr = new Temp();

               gen(t '=' E.addr '*' L.type.width); gen(L.addr '=' L1.addr '+' t); }
```

- 4.** In the L attributed definition, the attribute values of a node can inherit values from siblings on the left, is there any way to implement a right attributed definition where nodes will inherit values from the right siblings. 2
- CO4**
- 5.** If symbol table generation and scope analysis and type checking of expression in a program must be done in a single post-order traversal of the abstract syntax tree, how does it restrict the structure of the source program and why? 2
- CO2**

Please Turn Over

6. Consider the following Syntax Directed Translation (SDT) for generating three-address code for boolean expressions and control-flow statements using the fall-through technique (special label fall) to avoid redundant goto statements. 5
CO4 Draw the annotated parse tree and generate the final optimized three-address code produced by the SDT, including all necessary labels, for the following String: *if (x != 4 && y < 2) z = 0;*

Note : || means string concatenation in SDT rules. Here Temp() class creates a temporary variable which is 't:' with a subscript 1, 2, 3 and so on.

SDT:

```

P → S { S.next = 'fall'; P.code = S.code; }

S → id = E ; { S.code = E.code || gen( top.get(id.lexeme) " = " E.addr ); }

S → if ( B ) S1 { B.true = 'fall'; S1.next = S.next;
  if ( S.next == 'fall' ) { B.false = newLabel(); S.code = B.code || S1.code || gen( B.false " : " ); }
  else { B.false = S.next; S.code = B.code || S1.code; } }

B → B1 && B2
  { if ( B.false == 'fall' ) { B1.true = 'fall'; B1.false = newLabel(); B2.true = B.true;
    B2.false = 'fall'; B.code = B1.code || B2.code || gen( B1.false " : " ); }
  else { B1.true = 'fall'; B1.false = B.false; B2.true = B.true; B2.false = B.false;
    B.code = B1.code || B2.code; } }

B → E1 rel E2
  { B.code = E1.code || E2.code;
    if ( B.true == 'fall' ) {
      B.code = B.code || gen( "if " E1.addr " " not(rel.op) " " E2.addr " goto " B.false ); }
    else if ( B.false == 'fall' ) {
      B.code = B.code || gen( "if " E1.addr " " rel.op " " E2.addr " goto " B.true ); }
    else {
      B.code = B.code || gen( "if " E1.addr " " rel.op " " E2.addr " goto " B.true )
      || gen( "goto " B.false ); } }

E → id { E.addr = top.get(id.lexeme); E.code = ""; }

E → num { E.addr = newTemp(); E.code = gen( E.addr " = " num.val ); }

```

7. Draw an activation tree for these function call sequences and draw the state of the stack at the flow of execution at node f(3). 3
CO5

```

int b(int n)
{
    return n;
}
int f(int n)
{
    if ( n == 0 )
        return 0;
    return b(n)+f(n-1);
}
int main() //program will start from here
{
    int x;
    x = f ( 5 );
}

```

8. a. Consider the following Syntax Directed Translation (SDT). Draw the parse tree, show the attribute calculations and show the content of symbol tables for the input string: 5
CO5

```
float z;
record {
    int[1][3] z;
    int y;
} q;
int y;
```

SDT:

```
P → { offset = 0; top = new Env(); envStack = new Stack<Env>();
      offsetStack = new Stack<int>(); } D

D → T id ; { top.put(id.lexeme, T.type, offset); offset = offset + T.width; } D1

D → ε

T → B { C.inType = B.type; C.inWidth = B.width; } C { T.type = C.type; T.width = C.width; }

T → record '{' { envStack.push(top); top = new Env(); offsetStack.push(offset); offset = 0; }
      D '}' { Env recEnv = top; recWidth = offset; top = envStack.pop();
              offset = offsetStack.pop(); T.type = record(recEnv); T.width = recWidth; }

B → int { B.type = integer; B.width = 4; }

B → float { B.type = float; B.width = 8; }

C → ε { C.type = C.inType; C.width = C.inWidth; }

C → [ num ] C1 { C1.inType = C.inType; C1.inWidth = C.inWidth;
                  C.type = array(num.value, C1.type); C.width = num.value * C1.width; }
```

- b. For the same input string used in part (a), assume that the semantic actions for only the following two productions are updated, while all other grammar rules and semantic actions remain unchanged. Show the updated contents of all symbol tables 3

Given Helper Function:

```
int align(int n, int k) {
    if (n % k == 0) return n;
    else return n + (k - (n % k)); }
```

```
D → T id ; { if (offset % T.width != 0)
              offset = align(offset, T.width);
              top.put(id.lexeme, T.type, offset);
              offset = offset + T.width;
            } D1

T → record '{'
      { envStack.push(top); top = new Env();
        offsetStack.push(offset); offset = 0; }
      D '}'
      { Env recEnv = top; int recRawWidth = offset;
        top = envStack.pop(); offset = offsetStack.pop();
        T.type = record(recEnv);
        T.width = align(recRawWidth, 8); }
```